

UNIVERSIDAD DE GUADALAJARA

ARQUITECTURA DE SISTEMAS DE INTELIGENCIA ARTIFICIAL

CUGDL

Prerrequisitos
ASIA:
Machine Learning III

Mtro. David Echavarry

12 de marzo de 2026

Agenda

- Objetivo de clase
- Curvas de aprendizaje (Learning curves)
- Parámetros vs. Hiperparámetros
- Optimización en Machine Learning
- Teoría del aprendizaje
- Balance Ajuste–Simplicidad de Hipótesis
- Pérdida vs Métricas de Evaluación
- Desarrollo de Sistemas de Machine Learning
- Calidad de Datos y Features
- Monitoreo y Mantenimiento de Modelos

Objetivo de clase

Idea clave

- Concluir los **conceptos esenciales de Machine Learning** como prerequisites para un curso de ASIA.

Learning Curves

Curva de aprendizaje (Learning Curve)

Una **learning curve** describe cómo cambia el desempeño de un modelo de Machine Learning a medida que aumenta el tamaño del conjunto de entrenamiento.

- Sea un conjunto de datos:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

- Entrenamos un modelo utilizando subconjuntos crecientes de datos:

$$D_1 \subset D_2 \subset \dots \subset D_N$$

- Para cada tamaño del dataset medimos el desempeño del modelo mediante:

$$E_{train}(N) \quad \text{y} \quad E_{test}(N)$$

La **learning curve** muestra cómo cambian estos errores conforme aumenta el número de ejemplos utilizados para entrenar el modelo.

Learning Curves

Error de entrenamiento y error de prueba

Para analizar el desempeño del modelo se utilizan dos tipos de error:

- **Error de entrenamiento**

$$E_{train} = \frac{1}{N} \sum_{i=1}^N L(y_i, h(x_i))$$

Mide qué tan bien el modelo se ajusta a los datos utilizados durante el entrenamiento.

- **Error de prueba**

$$E_{test} = \frac{1}{M} \sum_{i=1}^M L(y_i, h(x_i))$$

Se calcula usando datos que el modelo **no ha visto durante el entrenamiento**.

Las learning curves muestran cómo evolucionan estos errores cuando aumenta el tamaño del conjunto de entrenamiento.

Interpretación de Learning Curves

Diagnóstico del modelo

Las learning curves permiten identificar diferentes comportamientos del modelo.

■ Underfitting

El modelo es demasiado simple para capturar los patrones de los datos.

$$E_{train} \text{ alto, } E_{test} \text{ alto}$$

■ Overfitting

El modelo memoriza los datos de entrenamiento y no generaliza bien.

$$E_{train} \text{ bajo, } E_{test} \text{ alto}$$

■ Buen ajuste

El modelo aprende patrones generales del problema.

$$E_{train} \approx E_{test}$$

Las learning curves permiten observar estos comportamientos a medida que aumenta el tamaño del dataset.

Learning Curves

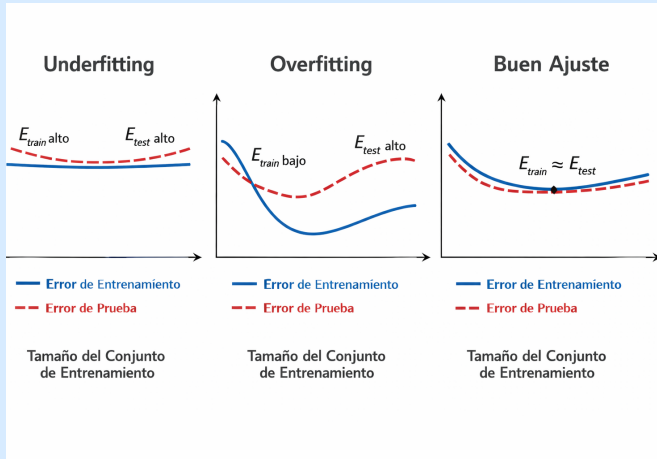
Interpretación de las curvas de aprendizaje

Las learning curves también permiten analizar la relación entre la cantidad de datos disponibles y la capacidad del modelo.

- Si el error de prueba disminuye significativamente al aumentar el tamaño del dataset, entonces **más datos pueden mejorar el desempeño del modelo**.
- Si el error de prueba se estabiliza y deja de disminuir, se dice que el modelo ha alcanzado una **saturación**.
- En este caso, agregar más datos probablemente no mejorará el desempeño.
- La relación entre:
 - complejidad del modelo
 - tamaño del dataset

determina si el modelo puede seguir mejorando al recibir más ejemplos.

Learning curves



Learning Curves

Modelo más complejo

En Machine Learning, la **complejidad del modelo** está relacionada con la capacidad del modelo para representar funciones más expresivas.

- Sea un modelo parametrizado:

$$h(x; \theta)$$

donde θ representa los parámetros del modelo.

- Un modelo más complejo suele tener:
 - mayor número de parámetros
 - mayor capacidad para representar funciones complejas
- Si un modelo presenta:

$$E_{train} \text{ alto, } E_{test} \text{ alto}$$

esto puede indicar que el modelo es demasiado simple.

- En este caso, utilizar un **modelo más complejo** puede permitir capturar mejor los patrones presentes en los datos.

Learning Curves

Ejemplo

Supongamos que queremos predecir el precio de una casa usando únicamente su tamaño.

- Un modelo muy simple podría ser una función lineal:

$$h(x) = w_0 + w_1x$$

- Si la relación real entre las variables es más compleja, el modelo puede producir errores altos tanto en entrenamiento como en prueba.
- Un modelo más complejo podría incluir términos adicionales:

$$h(x) = w_0 + w_1x + w_2x^2 + w_3x^3$$

- Este modelo tiene mayor capacidad para capturar relaciones no lineales entre las variables.

En este caso, aumentar la complejidad del modelo puede reducir el error de predicción.

Learning Curves

Regularización

La **regularización** es una técnica utilizada para controlar la complejidad del modelo y evitar el sobreajuste.

- En lugar de minimizar únicamente la pérdida:

$$\min_{\theta} L(\theta)$$

- Se introduce un término adicional que penaliza modelos demasiado complejos:

$$\min_{\theta} (L(\theta) + \lambda \Omega(\theta))$$

donde:

- $L(\theta)$ es la función de pérdida
- $\Omega(\theta)$ es una penalización sobre los parámetros
- λ controla la intensidad de la regularización
- La regularización favorece modelos más simples que generalizan mejor.

Learning Curves

Ejemplo

Supongamos que entrenamos un modelo muy complejo para predecir precios de casas.

- El modelo puede ajustarse perfectamente a los datos de entrenamiento:

$$E_{train} \approx 0$$

- Sin embargo, el error en datos nuevos puede ser alto:

$$E_{test} \text{ alto}$$

- Esto indica que el modelo está capturando ruido en los datos.
- Aplicar regularización penaliza parámetros muy grandes y favorece soluciones más simples.

Como resultado, el modelo puede tener un error de entrenamiento ligeramente mayor, pero una mejor capacidad de generalización.

Parámetros vs Hiperparámetros

Parámetros del modelo

Muchos modelos de Machine Learning pueden representarse como una función parametrizada:

$$h(x; \theta)$$

donde:

- x representa las variables de entrada.
- θ representa el conjunto de **parámetros del modelo**.
- $h(x; \theta)$ produce la predicción del modelo.

El proceso de entrenamiento consiste en encontrar los parámetros que minimizan una función de pérdida:

$$\theta^* = \arg \min_{\theta} L(\theta)$$

donde $L(\theta)$ mide el error del modelo sobre el conjunto de entrenamiento.

Parámetros vs Hiperparámetros

Ejemplo

Supongamos que queremos predecir el precio de una casa a partir de su tamaño.

- Un modelo lineal puede representarse como:

$$h(x) = w_0 + w_1x$$

- En este modelo:
 - w_0 es el intercepto.
 - w_1 es el coeficiente asociado a la variable de entrada.
- Los parámetros del modelo son:

$$\theta = (w_0, w_1)$$

Durante el entrenamiento, el algoritmo ajusta estos parámetros para minimizar el error de predicción.

Parámetros vs Hiperparámetros

Hiperparámetros

Los **hiperparámetros** son configuraciones del modelo que **no se aprenden directamente a partir de los datos**.

- Se definen **antes del entrenamiento** del modelo.
- Controlan aspectos del algoritmo de aprendizaje o de la estructura del modelo.
- Mientras los parámetros se obtienen resolviendo:

$$\theta^* = \arg \min_{\theta} L(\theta)$$

los hiperparámetros se seleccionan mediante procedimientos externos.
Estos valores determinan cómo se comporta el algoritmo durante el aprendizaje.

Parámetros vs Hiperparámetros

Ejemplo

Supongamos que entrenamos un modelo para clasificar correos electrónicos como spam o no spam.

- Algunos ejemplos de hiperparámetros pueden ser:
 - profundidad máxima de un modelo
 - número de vecinos considerados
 - tasa de aprendizaje de un algoritmo iterativo
 - coeficiente de regularización
- Estos valores **no se aprenden directamente** durante el entrenamiento.
- En su lugar, se prueban distintas configuraciones y se selecciona la que produce el mejor desempeño en datos de validación.

Parámetros vs Hiperparámetros

Espacio de configuraciones del modelo

El conjunto de posibles combinaciones de hiperparámetros define un **espacio de configuraciones** del modelo.

$$\Lambda = \{\lambda_1, \lambda_2, \dots, \lambda_k\}$$

donde cada λ_i representa un hiperparámetro.

- Cada configuración produce un modelo diferente.
- El objetivo de **model selection** es encontrar la configuración que mejor generaliza a datos no observados.
- Esto se logra evaluando múltiples configuraciones utilizando un conjunto de validación.

Parámetros vs Hiperparámetros

Aprendizaje paramétrico vs no paramétrico

Los modelos de Machine Learning pueden diferenciarse según cómo representan la información aprendida.

■ Modelos paramétricos

- Resumen los datos mediante un conjunto fijo de parámetros θ .
- El tamaño del modelo no depende del número de ejemplos de entrenamiento.

■ Modelos no paramétricos

- No resumen completamente los datos en un número fijo de parámetros.
- El modelo puede depender directamente del conjunto de entrenamiento.

Los hiperparámetros influyen directamente en la **complejidad del modelo** y en su capacidad para generalizar a nuevos datos.

Optimización en Machine Learning

Aprendizaje como problema de optimización

En muchos modelos de Machine Learning, el proceso de aprendizaje puede formularse como un problema de optimización.

- Sea un modelo parametrizado:

$$h(x; \theta)$$

- El objetivo del aprendizaje es encontrar los parámetros θ que produzcan las mejores predicciones.
- Esto se logra minimizando una función de pérdida:

$$\theta^* = \arg \min_{\theta} L(\theta)$$

- Donde $L(\theta)$ mide el error del modelo sobre el conjunto de entrenamiento.

El proceso de entrenamiento consiste en encontrar los parámetros que minimizan esta función.

Optimización en Machine Learning

Función objetivo

La **función objetivo** es la función que el algoritmo intenta optimizar durante el entrenamiento.

- En muchos problemas se define como la pérdida promedio sobre el conjunto de entrenamiento:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N L(y_i, h(x_i; \theta))$$

- donde:
 - N es el número de ejemplos de entrenamiento
 - y_i es el valor real
 - $h(x_i; \theta)$ es la predicción del modelo
- El objetivo del aprendizaje es encontrar los parámetros que minimizan esta función.

Optimización en Machine Learning

Superficie de error

La función de pérdida define una **superficie de error** sobre el espacio de parámetros.

- Cada punto en el espacio de parámetros θ corresponde a un valor de pérdida:

$$L(\theta)$$

- Esta función puede visualizarse como un paisaje o superficie.
- El objetivo del aprendizaje es encontrar el punto donde la pérdida es mínima.

$$\theta^* = \arg \min_{\theta} L(\theta)$$

Dependiendo de la forma de la superficie, el proceso de optimización puede ser más o menos difícil.

Optimización en Machine Learning

Mínimos locales y globales

En la superficie de error pueden existir distintos tipos de mínimos.

- **Mínimo global**

- Es el punto donde la función de pérdida alcanza su valor más bajo.

$$L(\theta^*) \leq L(\theta) \quad \forall \theta$$

- **Mínimo local**

- Es un punto donde la función es menor que en sus alrededores, pero no necesariamente el mínimo global.

En algunos modelos la superficie de error puede contener múltiples mínimos locales, lo que hace más difícil el proceso de optimización.

Optimización en Machine Learning

Métodos analíticos vs iterativos

Existen diferentes estrategias para resolver el problema de optimización.

■ Métodos analíticos

- Permiten encontrar una solución exacta mediante una fórmula cerrada.
- Se utilizan en algunos modelos simples.

■ Métodos iterativos

- Buscan mejorar los parámetros de manera progresiva.
- En cada iteración se actualizan los parámetros para reducir la función de pérdida.

Muchos algoritmos modernos de Machine Learning utilizan métodos iterativos de optimización.

Optimización en Machine Learning

Descenso por gradiente

Uno de los métodos iterativos más utilizados es el **gradient descent**.

- El gradiente indica la dirección de mayor incremento de la función de pérdida:

$$\nabla L(\theta)$$

- Para minimizar la pérdida, los parámetros se actualizan en la dirección opuesta:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

- donde:

- η es la tasa de aprendizaje
- $\nabla L(\theta_t)$ es el gradiente de la pérdida

El proceso continúa hasta que los cambios en los parámetros se vuelven pequeños, lo que indica **convergencia**.

Teoría del aprendizaje

Teoría del aprendizaje

La **teoría del aprendizaje** estudia las condiciones bajo las cuales un algoritmo puede aprender a partir de datos.

- Su objetivo es entender:
 - cuándo es posible aprender
 - cuántos datos se necesitan
 - cuánto cuesta computacionalmente aprender
- La teoría del aprendizaje analiza la relación entre:
 - el modelo utilizado
 - el tamaño del conjunto de datos
 - la dificultad del problema

En Machine Learning, aprender significa encontrar una hipótesis $h(x)$ que aproxime la función verdadera $f(x)$.

Sample Complexity

Complejidad muestral (Sample Complexity)

La **sample complexity** mide cuántos ejemplos de entrenamiento son necesarios para aprender una buena aproximación de la función verdadera.

- Sea un conjunto de entrenamiento:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

- La sample complexity analiza cuál es el tamaño mínimo N necesario para que el modelo generalice correctamente.
- En términos generales:

$$N \uparrow \Rightarrow \text{mejor estimación de } f(x)$$

Modelos más complejos generalmente requieren más datos para aprender correctamente.

Sample Complexity

Ejemplo

Supongamos que queremos aprender una función que clasifique correos electrónicos como spam o no spam.

- Si el modelo se entrena con pocos ejemplos:

$$N = 50$$

puede no capturar correctamente los patrones del problema.

- Si el modelo se entrena con muchos ejemplos:

$$N = 10000$$

es más probable que aprenda una aproximación más precisa de la función verdadera.

Esto ilustra cómo el número de ejemplos disponibles influye en la capacidad del modelo para aprender.

Computational Complexity

Complejidad computacional

La **computational complexity** mide el costo computacional necesario para entrenar y utilizar un modelo.

- Incluye dos componentes principales:

- **Costo de entrenamiento**

Tiempo y recursos necesarios para encontrar los parámetros óptimos:

$$\theta^* = \arg \min_{\theta} L(\theta)$$

- **Costo de predicción**

Tiempo necesario para calcular la salida del modelo:

$$\hat{y} = h(x; \theta)$$

Algunos algoritmos requieren más tiempo de entrenamiento o predicción que otros.

Complejidad del aprendizaje

Complejidad del modelo y aprendizaje

La dificultad del aprendizaje depende de varios factores.

- **Complejidad del modelo**

Modelos más expresivos pueden representar funciones más complejas, pero también pueden requerir más datos.

- **Cantidad de datos**

Un mayor número de ejemplos puede mejorar la estimación de la función verdadera.

- **Dificultad del problema**

Algunos problemas tienen estructuras simples, mientras que otros requieren modelos más complejos.

Existe una relación entre:

- complejidad del modelo
- tamaño del dataset
- dificultad del aprendizaje

Separabilidad de los datos

Problemas linealmente separables

Un problema es **linealmente separable** si existe una función lineal que separa correctamente las clases.

$$w^T x + b = 0$$

donde:

- w es un vector de pesos
- b es el intercepto
- Si los datos son separables, un modelo simple puede resolver el problema.
- Si los datos no son separables, se requieren modelos más complejos.

La estructura del problema influye directamente en la dificultad del aprendizaje.

Curse of Dimensionality

Maldición de la dimensionalidad

La **curse of dimensionality** describe los problemas que aparecen cuando el número de variables de entrada es muy grande.

- A medida que aumenta la dimensión del espacio de datos:
 - los datos se vuelven más dispersos
 - se requieren más ejemplos para aprender correctamente
 - la estimación de la función verdadera se vuelve más difícil
- En espacios de alta dimensión, muchos algoritmos necesitan cantidades mucho mayores de datos para generalizar correctamente.

Esto aumenta la dificultad del aprendizaje y el costo computacional del modelo.

Generalización y memorización

Generalización vs memorización

El objetivo del aprendizaje automático no es memorizar los datos de entrenamiento, sino aprender patrones que generalicen a nuevos datos.

- Sea la función verdadera del problema:

$$y = f(x)$$

- El modelo aprende una hipótesis:

$$\hat{y} = h(x)$$

- Si el modelo simplemente memoriza los ejemplos de entrenamiento, puede tener:

$$E_{train} \approx 0$$

pero un error alto en datos nuevos.

Un buen modelo debe capturar regularidades del problema y generalizar correctamente a ejemplos no observados.

Principio de simplicidad

Ockham's Razor en Machine Learning

El principio de **Ockham's Razor** establece que, entre múltiples explicaciones posibles, se debe preferir la más simple que explique los datos.

- En Machine Learning, esto implica preferir modelos con menor complejidad cuando producen un desempeño similar.
- Modelos excesivamente complejos pueden ajustarse al ruido presente en los datos.
- Modelos más simples tienden a generalizar mejor cuando capturan la estructura esencial del problema.

Este principio guía el diseño y la selección de modelos en muchos algoritmos de aprendizaje automático.

Espacio de hipótesis

Espacio de hipótesis

El **espacio de hipótesis** es el conjunto de todas las funciones que un algoritmo puede aprender.

$$H = \{h_1, h_2, \dots, h_n\}$$

- Cada hipótesis $h \in H$ representa un posible modelo para los datos.
- La **expresividad** del espacio de hipótesis determina qué tan complejas pueden ser las funciones representadas.
- Espacios de hipótesis más expresivos pueden representar funciones más complejas, pero también aumentan el riesgo de sobreajuste.

El aprendizaje consiste en seleccionar la hipótesis que mejor aproxima la función verdadera.

Complejidad del modelo

Complejidad y generalización

La complejidad del modelo influye directamente en su capacidad para generalizar.

- Modelos simples pueden no capturar patrones importantes en los datos.
- Modelos demasiado complejos pueden ajustarse al ruido presente en el conjunto de entrenamiento.
- Existe un equilibrio entre:
 - capacidad del modelo
 - número de datos disponibles
 - generalización

El objetivo es encontrar un modelo con suficiente complejidad para representar el problema, pero no tan complejo que memorice los datos.

Bias–Variance

Descomposición Bias–Variance

El error de un modelo puede descomponerse en diferentes componentes.

$$Error = Bias^2 + Variance + Noise$$

- **Bias**

Error causado por suposiciones demasiado simples en el modelo.

- **Variance**

Error causado por sensibilidad a variaciones en los datos de entrenamiento.

- **Noise**

Variabilidad inherente en los datos.

Modelos simples tienden a tener alto bias, mientras que modelos complejos pueden tener alta varianza.

Frontera de decisión

Frontera de decisión

En problemas de clasificación, el modelo divide el espacio de entrada mediante una **frontera de decisión**.

$$h(x) = 0$$

- La forma de la frontera depende de la complejidad del modelo.
- Fronteras simples corresponden a modelos más simples.
- Fronteras complejas pueden adaptarse mejor a los datos, pero también aumentar el riesgo de sobreajuste.

La simplicidad estructural de la hipótesis puede favorecer una mejor capacidad de generalización.

Control de complejidad

Regularización y combinación de hipótesis

Existen diferentes estrategias para controlar la complejidad del modelo.

■ Regularización

Introduce una penalización en la función objetivo para favorecer soluciones más simples.

$$\min_{\theta} (L(\theta) + \lambda \Omega(\theta))$$

■ Combinación de hipótesis

En algunos métodos se combinan múltiples modelos:

$$h(x) = \sum_{i=1}^k w_i h_i(x)$$

La combinación de hipótesis puede mejorar la capacidad de generalización al integrar la información de varios modelos.

Pérdida y métricas de evaluación

Error, función de pérdida y métrica

En Machine Learning es importante distinguir entre tres conceptos relacionados con el desempeño del modelo.

■ Error

Diferencia entre el valor real y la predicción del modelo.

$$error = y - \hat{y}$$

■ Función de pérdida

Función que mide qué tan incorrecta es una predicción individual.

$$L(y, \hat{y})$$

■ Métrica de evaluación

Medida agregada que resume el desempeño del modelo sobre un conjunto de datos.

$$Metric = \frac{1}{N} \sum_{i=1}^N L(y_i, \hat{y}_i)$$

Estos conceptos cumplen roles diferentes durante el entrenamiento y la evaluación de modelos.

Optimización durante el entrenamiento

Función objetivo

Durante el entrenamiento, el algoritmo intenta minimizar una función objetivo.

- En muchos modelos se optimiza la pérdida promedio:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N L(y_i, h(x_i; \theta))$$

- El proceso de aprendizaje consiste en encontrar:

$$\theta^* = \arg \min_{\theta} L(\theta)$$

- Es decir, el entrenamiento optimiza la **función de pérdida**, no necesariamente la métrica final utilizada para evaluar el modelo.

Evaluación del modelo

Métricas de evaluación

Después del entrenamiento, el modelo se evalúa utilizando métricas que resumen su desempeño.

- Algunas métricas comunes incluyen:

- exactitud
- error medio
- precisión
- recall
- F1-score

- Estas métricas se calculan sobre un conjunto de datos de prueba.

El objetivo es estimar qué tan bien el modelo generaliza a datos que no fueron utilizados durante el entrenamiento.

Costos de error

Costos de error en distintos contextos

No todos los errores tienen el mismo impacto en aplicaciones reales.

- En algunos problemas, ciertos errores son más costosos que otros.
- Por ejemplo:
 - En diagnóstico médico, un falso negativo puede ser más grave que un falso positivo.
 - En detección de fraude, no detectar una transacción fraudulenta puede ser muy costoso.
- Por esta razón, la función de pérdida puede diseñarse para reflejar estos costos.

El diseño de la función de pérdida debe considerar el contexto del problema.

Funciones de pérdida

Penalización de errores grandes

Algunas funciones de pérdida penalizan los errores grandes con mayor intensidad.

- Un ejemplo es el error cuadrático:

$$L(y, \hat{y}) = (y - \hat{y})^2$$

- En esta función, los errores grandes producen penalizaciones mucho mayores.
- Esto puede ayudar al modelo a reducir predicciones muy incorrectas.

La elección de la función de pérdida influye directamente en el comportamiento del modelo durante el entrenamiento.

Función objetivo y regularización

Función objetivo completa

En muchos modelos, la función optimizada incluye un término de regularización.

$$\min_{\theta} (L(\theta) + \lambda\Omega(\theta))$$

donde:

- $L(\theta)$ es la función de pérdida
- $\Omega(\theta)$ penaliza modelos demasiado complejos
- λ controla la intensidad de la regularización

La función de pérdida utilizada durante el entrenamiento debe estar alineada con el **objetivo real del problema**.

Sistemas de Machine Learning

Machine Learning como sistema

Un sistema de Machine Learning no consiste únicamente en entrenar un modelo.

- Un sistema completo incluye múltiples componentes:
 - recolección de datos
 - procesamiento de datos
 - entrenamiento del modelo
 - evaluación
 - despliegue en producción
- El modelo es solo una parte dentro de un sistema más amplio que procesa datos y produce predicciones.

El desempeño final de un sistema de Machine Learning depende de la interacción entre todos estos componentes.

Pipeline de Machine Learning

Pipeline general del aprendizaje automático

El desarrollo de sistemas de Machine Learning suele seguir un pipeline de etapas.

Datos → *Features* → *Modelo* → *Predicción*

- Cada etapa transforma la información disponible para producir mejores predicciones.
- El desempeño final del sistema depende de la calidad de cada una de estas etapas.

Este proceso puede representarse como una cadena de transformaciones que convierten datos en decisiones.

Datos y características

Recolección de datos e ingeniería de features

Las primeras etapas del desarrollo de sistemas de Machine Learning se enfocan en los datos.

■ Recolección de datos

Obtención de ejemplos representativos del problema.

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

■ Ingeniería de features

Transformación de los datos originales en representaciones útiles para el modelo.

$$x \rightarrow \phi(x)$$

La calidad de los datos y las características utilizadas tiene un impacto directo en el desempeño del modelo.

Entrenamiento y validación

Entrenamiento del modelo

Durante el entrenamiento se ajustan los parámetros del modelo para minimizar la función de pérdida.

$$\theta^* = \arg \min_{\theta} L(\theta)$$

- Los datos suelen dividirse en diferentes subconjuntos:
 - entrenamiento
 - validación
 - prueba
- La validación permite seleccionar modelos y configuraciones que generalicen mejor a datos nuevos.

Despliegue de modelos

Despliegue en producción

Una vez entrenado, el modelo puede integrarse en un sistema que produzca predicciones en tiempo real.

- En esta etapa se consideran aspectos como:
 - escalabilidad del sistema
 - costo computacional del modelo
 - eficiencia de inferencia
- La predicción del modelo se calcula como:

$$\hat{y} = h(x; \theta)$$

En aplicaciones reales, el modelo debe ser capaz de producir predicciones de manera eficiente para grandes volúmenes de datos.

Reproducibilidad

Reproducibilidad del entrenamiento

La reproducibilidad es un aspecto importante en el desarrollo de sistemas de Machine Learning.

- Permite obtener los mismos resultados al repetir el proceso de entrenamiento.
- Para lograr reproducibilidad es necesario controlar factores como:
 - versiones de datos
 - configuraciones del modelo
 - hiperparámetros
 - procedimientos de entrenamiento

La reproducibilidad facilita la validación de resultados y el mantenimiento de sistemas de Machine Learning.

Calidad de datos en Machine Learning

Importancia de los datos

El desempeño de un sistema de Machine Learning depende en gran medida de la calidad de los datos utilizados.

- Los modelos aprenden patrones a partir de ejemplos:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

- Si los datos contienen errores, sesgos o información incompleta, el modelo puede aprender patrones incorrectos.
- La calidad de los datos influye directamente en la capacidad del modelo para generalizar a nuevos ejemplos.

En muchos problemas prácticos, mejorar los datos puede tener un impacto mayor que cambiar el algoritmo de aprendizaje.

Representación de features

Representación de características

En Machine Learning, cada ejemplo se representa mediante un conjunto de características o **features**.

$$x = (x_1, x_2, \dots, x_d)$$

donde:

- d representa el número de variables de entrada.
- cada componente describe una propiedad del ejemplo.

La forma en que se representan las características determina qué patrones puede aprender el modelo.

Espacio de entrada

Espacio de características

El conjunto de todas las posibles representaciones de entrada define el **espacio de características**.

$$X \subseteq \mathbb{R}^d$$

- Cada ejemplo corresponde a un punto en este espacio.
- Los algoritmos de Machine Learning buscan patrones en la distribución de estos puntos.

La estructura del espacio de entrada influye en la dificultad del problema de aprendizaje.

Problemas en los datos

Datos ruidosos e incompletos

En muchos problemas reales, los datos disponibles presentan imperfecciones.

- **Datos ruidosos**

- contienen errores de medición o variaciones aleatorias.

- **Datos incompletos**

- algunos valores de las variables pueden faltar.

- **Sesgo en los datos**

- los datos pueden no representar adecuadamente la población real.

Estos problemas pueden afectar la capacidad del modelo para aprender correctamente.

Ingeniería de features

Ingeniería de características

La **ingeniería de features** consiste en transformar los datos originales para facilitar el aprendizaje.

- Las transformaciones pueden representarse como:

$$\phi(x)$$

donde ϕ es una función que genera una nueva representación de los datos.

- Una buena representación puede hacer que patrones complejos se vuelvan más fáciles de aprender.

La selección y transformación de características es una parte fundamental del desarrollo de modelos de Machine Learning.

Similitud entre ejemplos

Distancia en el espacio de features

Muchos algoritmos de Machine Learning utilizan medidas de distancia para comparar ejemplos.

- Una medida común es la distancia euclidiana:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^d (x_{ik} - x_{jk})^2}$$

- Ejemplos cercanos en el espacio de características suelen compartir propiedades similares.

La noción de similitud entre ejemplos es fundamental para muchos algoritmos de aprendizaje.

Dimensionalidad y ruido

Maldición de la dimensionalidad

Cuando el número de características aumenta, el espacio de datos se vuelve cada vez más grande.

- Los datos se vuelven más dispersos.
- Se necesitan más ejemplos para aprender correctamente.
- Las medidas de distancia pueden volverse menos informativas.

Además, el error total del modelo puede descomponerse como:

$$Error = Bias^2 + Variance + Noise$$

donde el término de **noise** representa la variabilidad inherente en los datos.

Modelos en producción

Modelos en producción

Después del entrenamiento, un modelo de Machine Learning puede integrarse en un sistema que realice predicciones en tiempo real.

- La predicción del modelo se calcula como:

$$\hat{y} = h(x; \theta)$$

- En producción, el modelo recibe continuamente nuevos datos de entrada.
- Las predicciones del modelo pueden utilizarse para apoyar decisiones automáticas o asistidas.

El comportamiento del modelo debe ser monitoreado continuamente para asegurar que mantenga un buen desempeño.

Degradación del modelo

Degradación del modelo

El desempeño de un modelo puede deteriorarse con el tiempo.

- Un modelo entrenado en un conjunto de datos:

$$D_{train}$$

puede enfrentar nuevas distribuciones de datos en producción.

- Cuando la distribución de los datos cambia, el modelo puede producir predicciones menos precisas.

Este fenómeno se conoce como **degradación del modelo**.

Cambios en los datos

Data Drift y Concept Drift

Existen distintos tipos de cambios en los datos que pueden afectar el desempeño del modelo.

■ Data Drift

- ocurre cuando cambia la distribución de las variables de entrada:

$$P_{train}(x) \neq P_{production}(x)$$

■ Concept Drift

- ocurre cuando cambia la relación entre las variables de entrada y la salida:

$$P(y|x)$$

Estos cambios pueden provocar que un modelo previamente preciso deje de funcionar correctamente.

Monitoreo del modelo

Monitoreo de desempeño

Los sistemas de Machine Learning deben monitorear continuamente el desempeño del modelo.

- Algunas métricas utilizadas pueden incluir:
 - exactitud
 - error medio
 - precisión
 - recall
- El monitoreo permite detectar:
 - degradación del modelo
 - cambios en la distribución de los datos

Detectar estos problemas a tiempo permite tomar acciones correctivas.

Actualización del modelo

Actualización del modelo

Cuando el desempeño del modelo disminuye, puede ser necesario actualizarlo.

- Algunas estrategias incluyen:
 - reentrenar el modelo con nuevos datos
 - ajustar hiperparámetros
 - cambiar la arquitectura del modelo
- El nuevo modelo se obtiene mediante un nuevo proceso de entrenamiento:

$$\theta^* = \arg \min_{\theta} L(\theta)$$

La actualización permite que el sistema se adapte a cambios en el entorno.

Gestión de modelos

Versionado y rollback

En sistemas de Machine Learning es importante mantener control sobre las versiones del modelo.

- **Versionado**

- cada modelo entrenado se almacena como una versión distinta.

- **Rollback**

- si una nueva versión produce resultados incorrectos, el sistema puede regresar a una versión anterior.

Estos mecanismos permiten mantener sistemas de Machine Learning confiables en entornos cambiantes.

Resumen del proceso de aprendizaje

Aprender a partir de datos

El objetivo del aprendizaje automático es aproximar una función desconocida utilizando ejemplos.

$$y = f(x)$$

A partir de un conjunto de datos:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

el algoritmo selecciona una hipótesis del espacio de hipótesis:

$$h(x) \in H$$

tal que la predicción del modelo:

$$\hat{y} = h(x)$$

aproxime lo mejor posible la función verdadera.

Componentes del aprendizaje

Componentes fundamentales

El aprendizaje automático combina varios elementos fundamentales.

- **Datos**

Ejemplos utilizados para aprender patrones.

- **Espacio de hipótesis**

Conjunto de funciones que el modelo puede representar.

- **Función de pérdida**

Mide qué tan incorrectas son las predicciones.

$$L(y, \hat{y})$$

- **Optimización**

Proceso de encontrar los parámetros que minimizan la pérdida.

$$\theta^* = \arg \min_{\theta} L(\theta)$$

Estos componentes trabajan conjuntamente para producir un modelo que generalice a nuevos datos.

Generalización y complejidad

Equilibrio en el aprendizaje

Un desafío central del aprendizaje automático es encontrar el equilibrio adecuado entre complejidad y generalización.

- Modelos demasiado simples pueden producir **underfitting**.
- Modelos demasiado complejos pueden producir **overfitting**.
- La capacidad del modelo debe ser adecuada para:
 - la cantidad de datos disponibles
 - la complejidad del problema

El objetivo es encontrar una hipótesis que capture la estructura del problema sin memorizar los datos.

Sistemas de Machine Learning

Machine Learning en la práctica

En aplicaciones reales, Machine Learning forma parte de sistemas más amplios.

- recolección de datos
- representación de características
- entrenamiento del modelo
- evaluación
- despliegue en producción
- monitoreo y actualización

El éxito de un sistema de Machine Learning depende tanto del modelo como de la calidad de los datos y del diseño del sistema completo.

Hyperparameter Tuning

Ajuste de hiperparámetros

El **hyperparameter tuning** consiste en buscar configuraciones de hiperparámetros que produzcan el mejor desempeño del modelo.

- Los hiperparámetros controlan el comportamiento del algoritmo de aprendizaje.
- Diferentes configuraciones pueden producir modelos con distinto desempeño.

El objetivo es encontrar:

$$\lambda^* = \arg \max_{\lambda} Metric(\lambda)$$

donde λ representa el conjunto de hiperparámetros.

Búsqueda de hiperparámetros

Métodos de búsqueda

Existen diferentes estrategias para explorar el espacio de hiperparámetros.

- **Grid Search**

Evalúa todas las combinaciones posibles de hiperparámetros.

- **Random Search**

Evalúa configuraciones aleatorias del espacio de hiperparámetros.

- **Optimización bayesiana**

Utiliza modelos probabilísticos para seleccionar configuraciones prometedoras.

Estos métodos permiten mejorar el desempeño del modelo sin cambiar su estructura.

Ensemble Learning

Combinación de modelos

El **ensemble learning** consiste en combinar múltiples modelos para producir mejores predicciones.

- Cada modelo produce una hipótesis:

$$h_1(x), h_2(x), \dots, h_k(x)$$

- El modelo final combina estas predicciones:

$$h(x) = \sum_{i=1}^k w_i h_i(x)$$

La combinación de modelos puede reducir errores y mejorar la capacidad de generalización.

Tipos de ensembles

Métodos de combinación

Existen distintas estrategias para combinar modelos.

- **Bagging**

Entrena múltiples modelos usando diferentes subconjuntos de datos.

- **Boosting**

Entrena modelos secuencialmente enfocándose en los errores anteriores.

- **Voting**

Combina predicciones mediante votación.

- **Stacking**

Utiliza un modelo adicional para combinar las predicciones de otros modelos.

Estos métodos permiten mejorar la estabilidad y precisión de los sistemas de Machine Learning.

Idea fundamental

Principio de los ensembles

La idea fundamental de los ensembles es que múltiples modelos pueden capturar diferentes aspectos de los datos.

- Un modelo individual puede cometer errores sistemáticos.
- La combinación de varios modelos puede compensar estos errores.

Esto permite obtener predicciones más robustas y estables.

Tres pilares del Machine Learning

Los tres pilares del aprendizaje automático

El desarrollo de modelos de Machine Learning puede entenderse como la interacción de tres elementos fundamentales.

■ Datos

Conjunto de ejemplos utilizados para aprender patrones.

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

■ Modelo

Espacio de hipótesis que define las funciones que el sistema puede aprender.

$$h(x; \theta)$$

■ Optimización

Proceso mediante el cual se ajustan los parámetros del modelo.

$$\theta^* = \arg \min_{\theta} L(\theta)$$

El éxito del aprendizaje automático depende del equilibrio entre estos tres componentes.

Interacción entre los pilares

Interacción entre datos, modelos y optimización

Los tres pilares del Machine Learning están estrechamente relacionados.

- **Datos**

determinan qué patrones pueden aprenderse.

- **Modelos**

definen la complejidad de las funciones que pueden representarse.

- **Optimización**

permite encontrar los parámetros que minimizan la función de pérdida.

Un sistema de Machine Learning exitoso requiere:

- datos representativos
- modelos adecuados
- métodos de optimización eficientes

Perspectiva general

Perspectiva general del aprendizaje automático

El aprendizaje automático puede interpretarse como el proceso de encontrar una función que generalice a partir de datos.

$$f(x) \approx h(x)$$

- los datos proporcionan ejemplos del problema
- el modelo define el espacio de soluciones posibles
- la optimización selecciona la mejor solución dentro de ese espacio

Este marco conceptual permite comprender una gran variedad de algoritmos de Machine Learning.

Bibliografía

Referencias

- Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.